



Department of Artificial Intelligence & Machine Learning
Academic Year 2025-26(EVEN) Report for
22AIM672 API and Microservices
On
Secure User & Product Management System Using Microservices
By

NAME	USN
Vishal Sagar P	1NH23AI179
Vishnu	1NH23AI180

Dept. of Artificial Intelligence & Machine Learning, New
Horizon College of Engineering,
Bangalore-560103

PROBLEM STATEMENT

In modern web applications, secure and efficient management of user and product information is essential. Traditional systems often use monolithic architecture, where all functionalities are combined into a single application. Such systems are difficult to maintain, scale, and secure, and they often lack proper authentication and centralized request handling.

Without secure access control, user and product operations become vulnerable to unauthorized access. Managing multiple functionalities within one system also increases complexity and reduces flexibility. In addition, the absence of centralized API routing makes communication between client and server less efficient and harder to manage.

To overcome these challenges, there is a need for a secure and modular system that separates functionalities into independent services while ensuring secure communication. The system should provide authentication, authorization, centralized routing, and efficient CRUD operations.

To address these issues, the proposed project titled “**Secure User & Product Management System using Microservices**” is developed using Node.js, Express.js, JWT Authentication, NGINX API Gateway, HTML, CSS, and JavaScript. The system provides secure login authentication, token-based access, centralized API routing, and efficient user and product management through microservices architecture.

Additionally, many existing systems do not provide proper modularity and secure token-based communication between services, which can lead to poor scalability and increased maintenance complexity. In large-scale applications, handling authentication, request routing, and CRUD operations within a single system can reduce performance and create security vulnerabilities. Therefore, implementing a microservices-based solution with JWT authentication and centralized API routing helps in improving system security, flexibility, maintainability, and efficient communication between frontend and backend components.

INTRODUCTION

The rapid growth of web technologies has increased the need for secure and scalable systems for managing data efficiently. In many applications, managing user and product information is an essential task that requires proper authentication, secure communication, and efficient request handling. Traditional monolithic systems often face difficulties in maintenance, scalability, and security due to tightly coupled components.

To overcome these limitations, microservices architecture provides a better approach by dividing the application into smaller independent services. Each service handles a specific functionality, making the system easier to maintain, scale, and manage.

The project titled “**Secure User & Product Management System using Microservices**” is developed as a web-based application that demonstrates secure user authentication and product management using microservices architecture. The system uses **JWT Authentication** for secure login and authorization, ensuring that only authenticated users can access protected resources.

The project is developed using **Node.js, Express.js, HTML, CSS, JavaScript, and NGINX API Gateway**. The frontend provides an interactive interface for login and dashboard operations, while the backend consists of separate microservices for authentication, user management, and product management.

The system also implements CRUD operations that allow users to create, view, update, and delete user and product records efficiently. NGINX acts as a centralized API Gateway that routes client requests to the appropriate microservices, improving system organization and security.

The main objective of this project is to provide a secure, efficient, and modular platform for performing CRUD operations on users and products while demonstrating practical implementation of API integration, authentication mechanisms, and microservices architecture used in modern web development.

Additionally, the project enhances understanding of modern web application architecture by demonstrating communication between frontend and backend services through REST APIs. The use of JWT-based authentication improves security by protecting sensitive user operations and preventing unauthorized access. The separation of functionalities into independent microservices increases system reliability, flexibility, and maintainability. This project also provides practical exposure to API Gateway implementation, secure token handling, and CRUD-based data management, making it suitable for real-world scalable web applications and modern software development practices.

CODE EXPLANATION

Backend Implementation:

The backend of the **Secure User & Product Management System using Microservices** is developed using **Node.js, Express.js, and NGINX API Gateway**. The backend handles user management, product management, API routing, and secure communication between services.

The backend starts by importing required modules.

Importing Required Modules:

```
const express = require('express');  
const cors = require('cors');
```

Express

- Express is a lightweight Node.js framework.
- It is used for creating REST APIs.
- It helps in handling HTTP requests like GET, POST, PUT, and DELETE.

CORS

- CORS stands for Cross-Origin Resource Sharing.
- It allows communication between frontend and backend running on different ports.
- Without CORS, browser security policies block requests.

Creating Express Application:

```
const app = express();  
app.use(cors());  
app.use(express.json());
```

This code performs:

- Creates Express application
- Enables cross-origin requests
- Parses incoming JSON data

User Service

The User Service handles CRUD operations for users.

```
let users = [  
  {id:1,name:"name"},  
  {id:2,name:"Admin"}  
];
```

This stores user records.

Get Users API

```
app.get('/users',(req,res)=>{  
  res.json(users);  
});
```

This API:

- Receives request
- Fetches all users
- Returns user data in JSON format

Add User API

```
app.post('/users',(req,res)=>{  
  const newUser = {  
    id: users.length+1,  
    name: req.body.name  
  };  
  
  users.push(newUser);  
  res.json(newUser);  
});
```

This API performs:

- Receives user details
- Creates new user
- Stores user data
- Sends response

Update User API:

```
app.put('/users/:id',(req,res)=>{  
  const user = users.find(  
    u => u.id === req.params.id  
  );  
  
  if(user){  
    user.name = req.body.name;  
    res.json(user);  
  }  
});
```

This API updates existing user information.

Delete User API

```
app.delete('/users/:id',(req,res)=>{  
  users = users.filter(  
    u => u.id !== req.params.id  
  );  
  
  res.send("User Deleted");  
});
```

This API removes selected user.

Product Service

The Product Service handles product management.

```
let products = [  
  {id:1,name:"Laptop"},  
  {id:2,name:"Mobile"}  
];
```

Get Products API

```
app.get('/products',(req,res)=>{  
  res.json(products);  
});
```

This retrieves all product details.

Add Product API

```
app.post('/products',(req,res)=>{  
  const newProduct = {  
    id: products.length+1,  
    name:req.body.name  
  };  
  
  products.push(newProduct);  
  res.json(newProduct);  
});
```

This adds a new product.

Update Product API

```
app.put('/products/:id',(req,res)=>{  
  const product = products.find(  
    p=>p.id==req.params.id  
  );  
  
  if(product){  
    product.name=req.body.name;  
    res.json(product);  
  }  
});
```

This updates product details.

Delete Product API

```
app.delete('/products/:id',(req,res)=>{  
  products = products.filter(  
    p=>p.id!=req.params.id  
  );  
  
  res.send("Product Deleted");  
});
```

This removes product data.

NGINX API Gateway

NGINX acts as the central gateway for routing client requests.

NGINX Configuration

```
server {  
    listen 8080;  
  
    location /api/users {  
        proxy_pass http://localhost:3001/users;  
    }  
  
    location /api/products {  
        proxy_pass http://localhost:3002/products;  
    }  
}
```

This configuration routes:

- User requests → User Service
- Product requests → Product Service

NGINX simplifies communication between frontend and backend services.

Frontend Implementation

The frontend is developed using **HTML, CSS, and JavaScript**.

Login Page

```
<input type="text" id="username">  
<input type="password" id="password">  
<button onclick="login()">Login</button>
```

This page allows users to log in.

Dashboard Buttons

```
<button onclick="getUsers()">View Users</button>  
<button onclick="addUser()">Add User</button>  
<button onclick="updateUser()">Update User</button>  
<button onclick="deleteUser()">Delete User</button>  
<button onclick="getProducts()">View Products</button>  
<button onclick="addProduct()">Add Product</button>  
<button onclick="updateProduct()">Update Product</button>  
<button onclick="deleteProduct()">Delete Product</button>
```

These buttons provide CRUD operations.

JavaScript Fetch API

Fetching Users

```
fetch("http://localhost:8080/api/users")
.then(res=>res.json())
.then(data=>{
  document.getElementById("output")
  .innerHTML=JSON.stringify(data);
});
```

This code:

1. Sends request to NGINX gateway
2. Receives user data
3. Displays output

Adding New User

```
fetch("http://localhost:8080/api/users",{
  method:"POST",
  headers:{
    "Content-Type":"application/json"
  },
  body:JSON.stringify({
    name:"New User"
  })
});
```

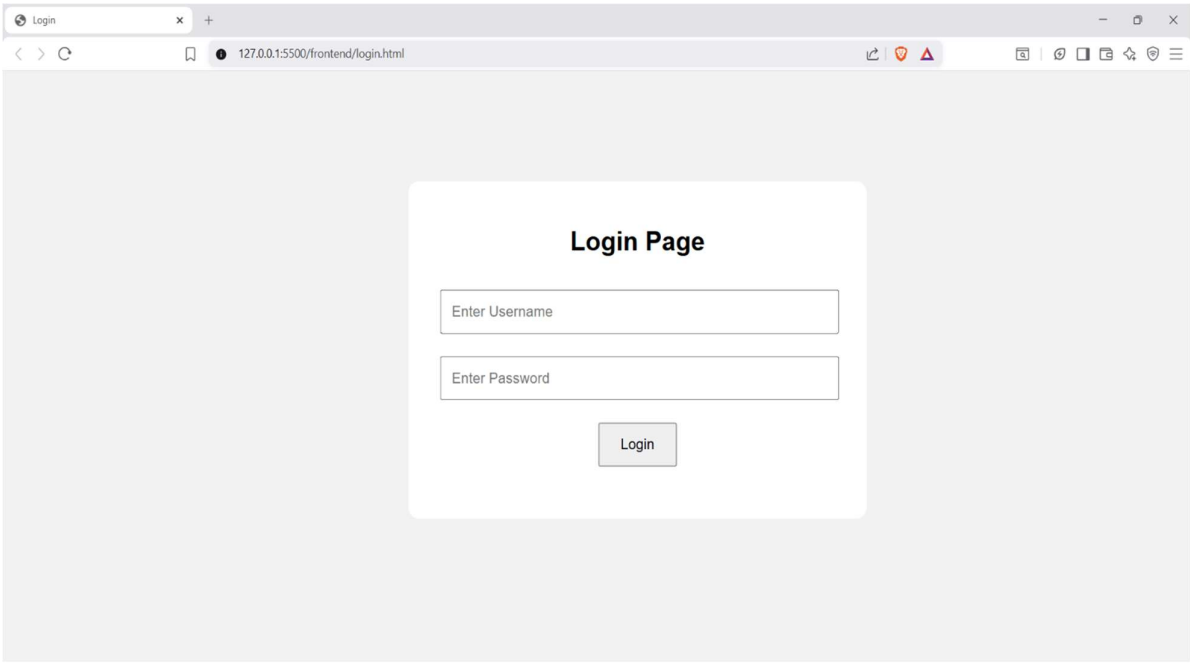
This sends user data to backend.

Application Execution

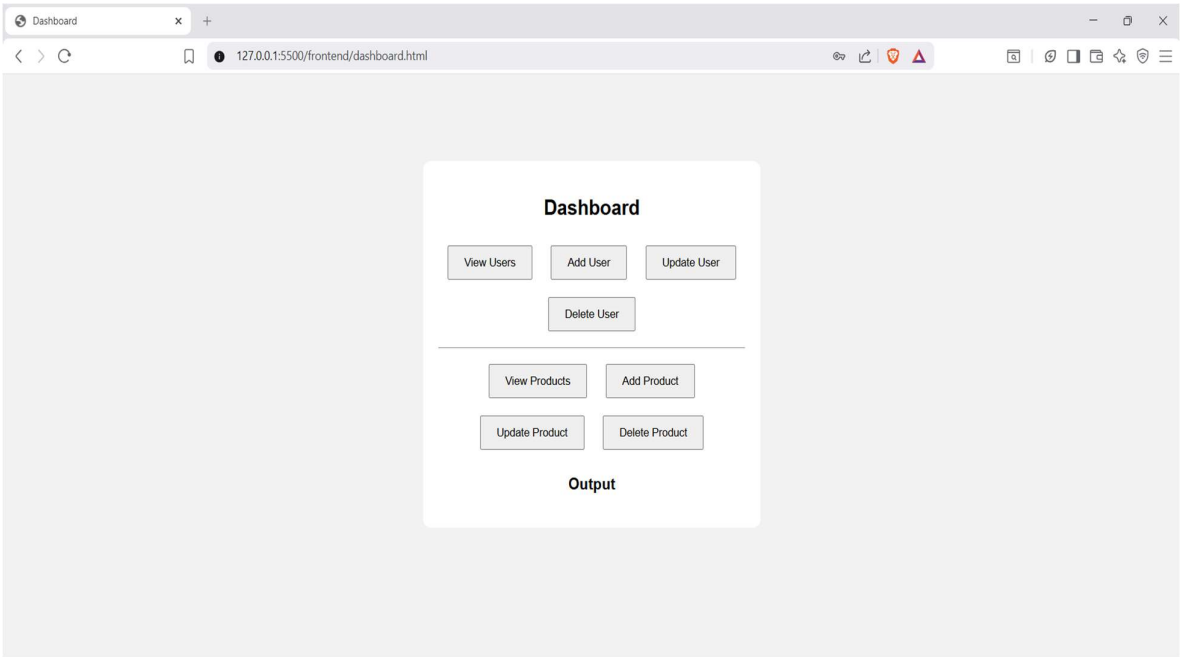
The project runs on the following ports:

- **Frontend:** 80
- **User Service:** 3001
- **Product Service:** 3002
- **NGINX Gateway:** 8080

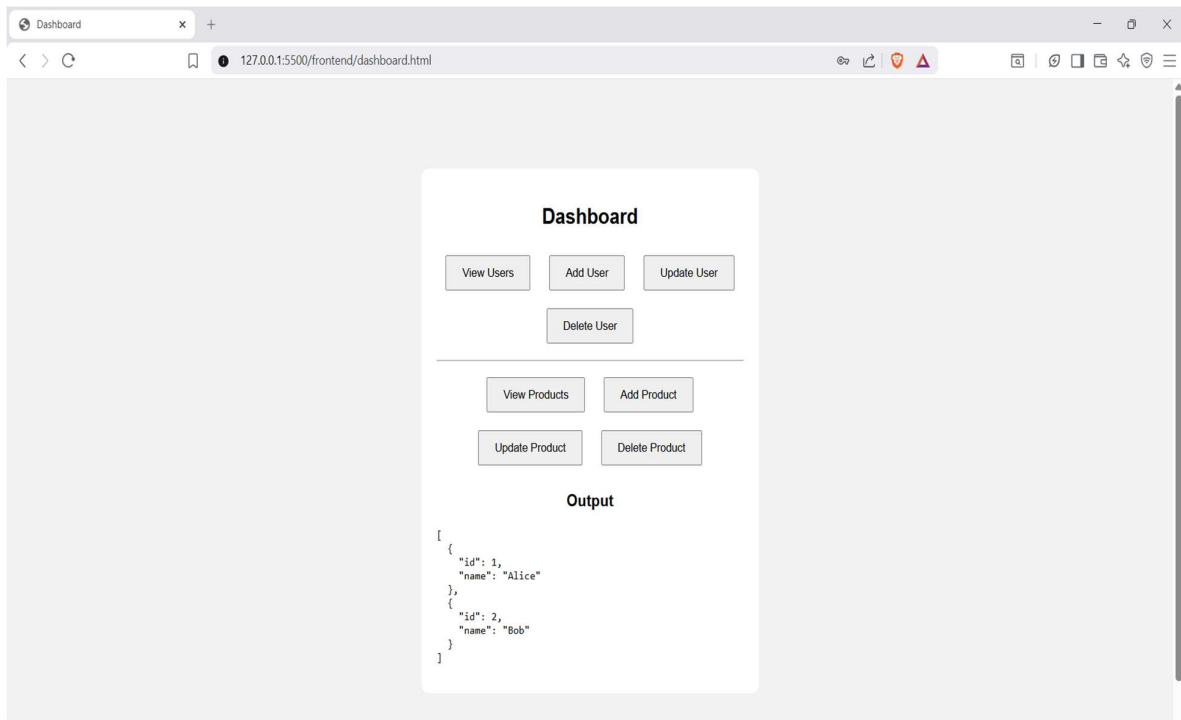
OUTPUTS



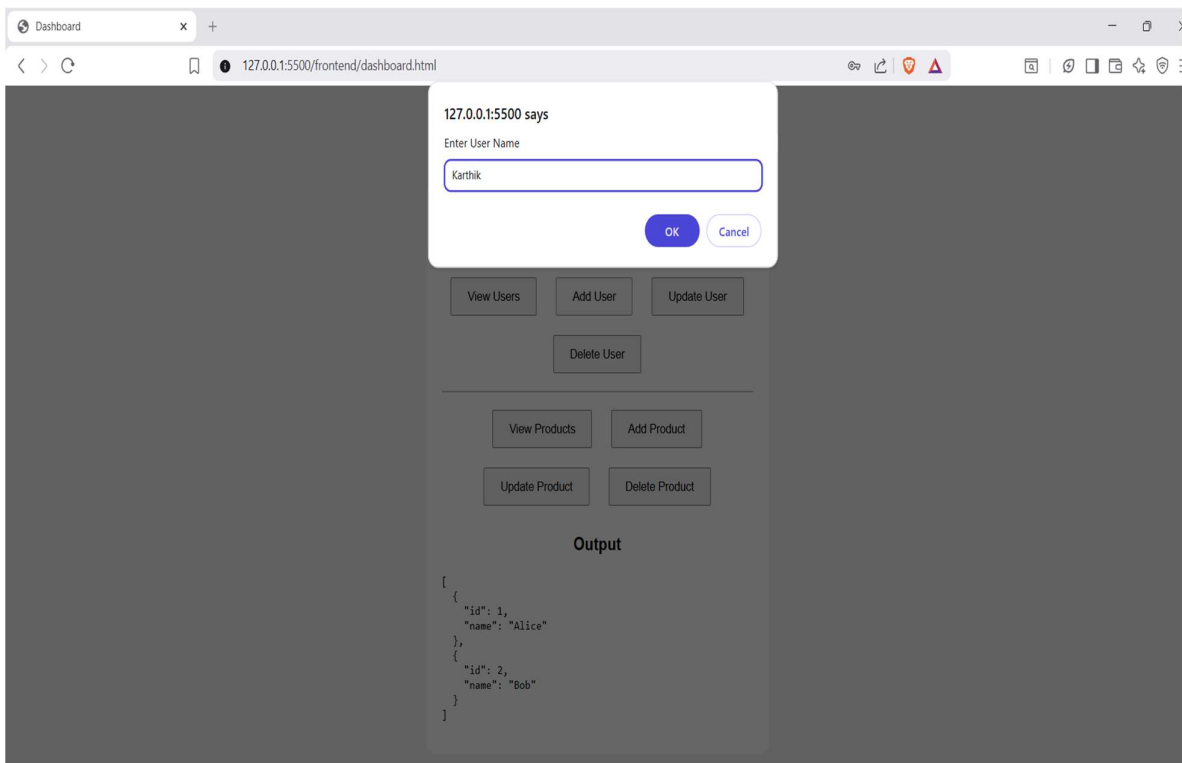
Login Page



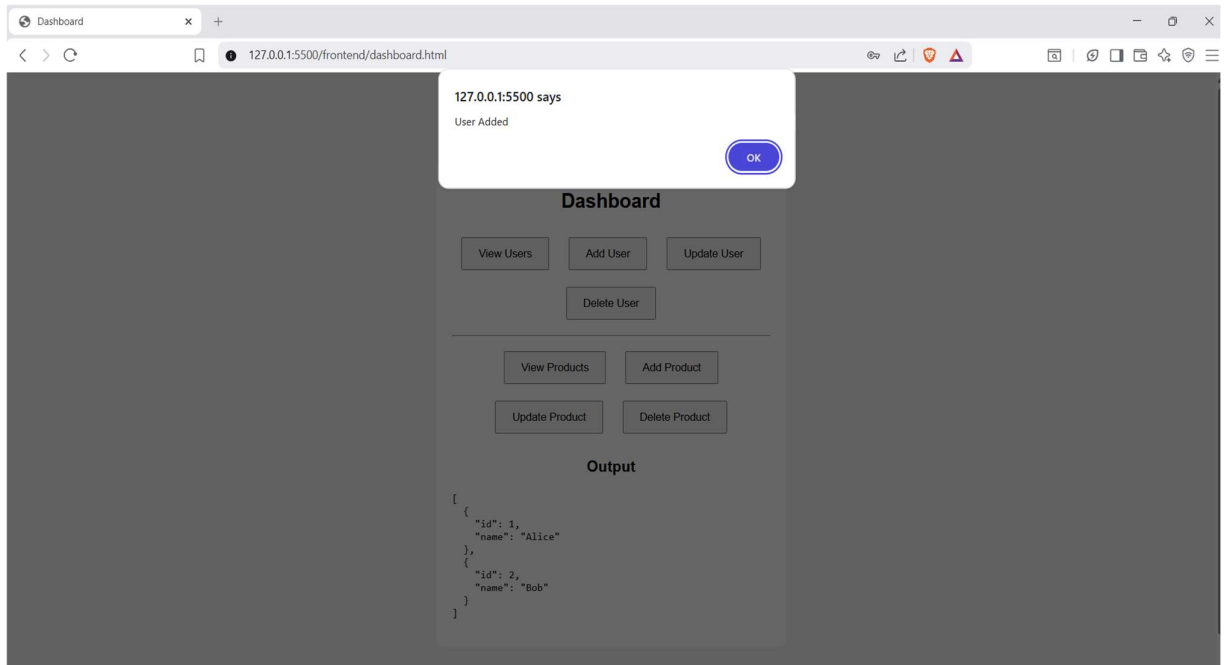
DashBoard



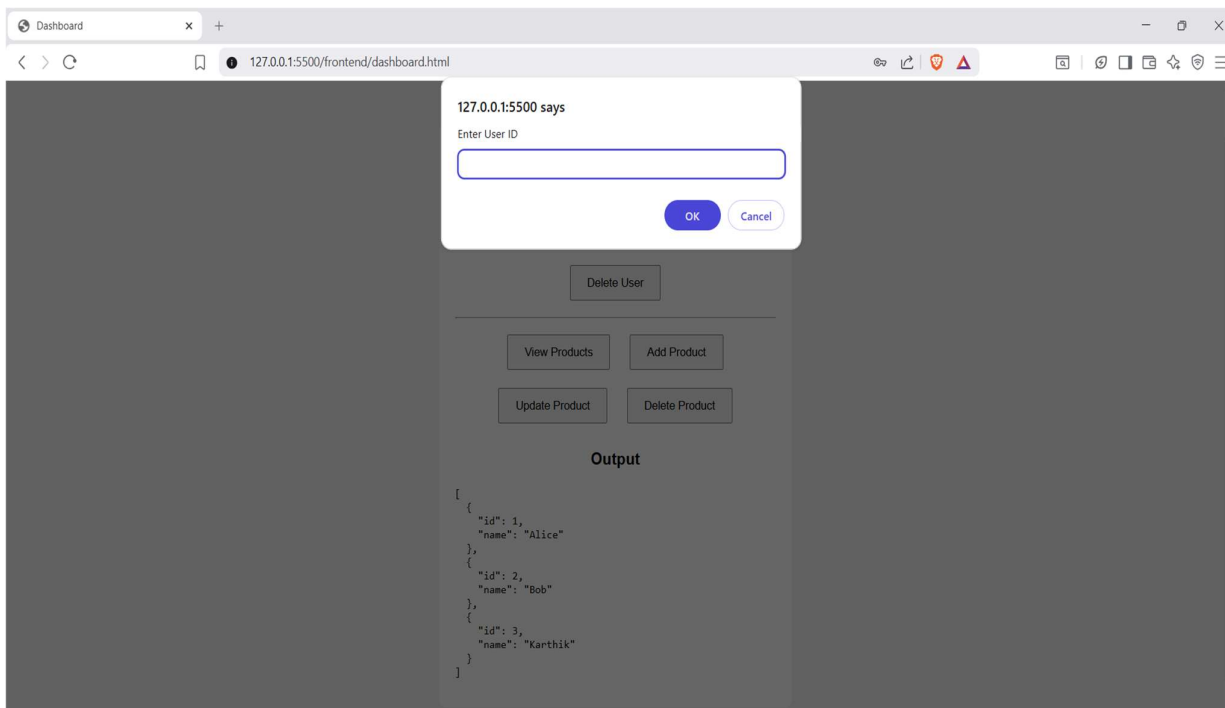
View Users



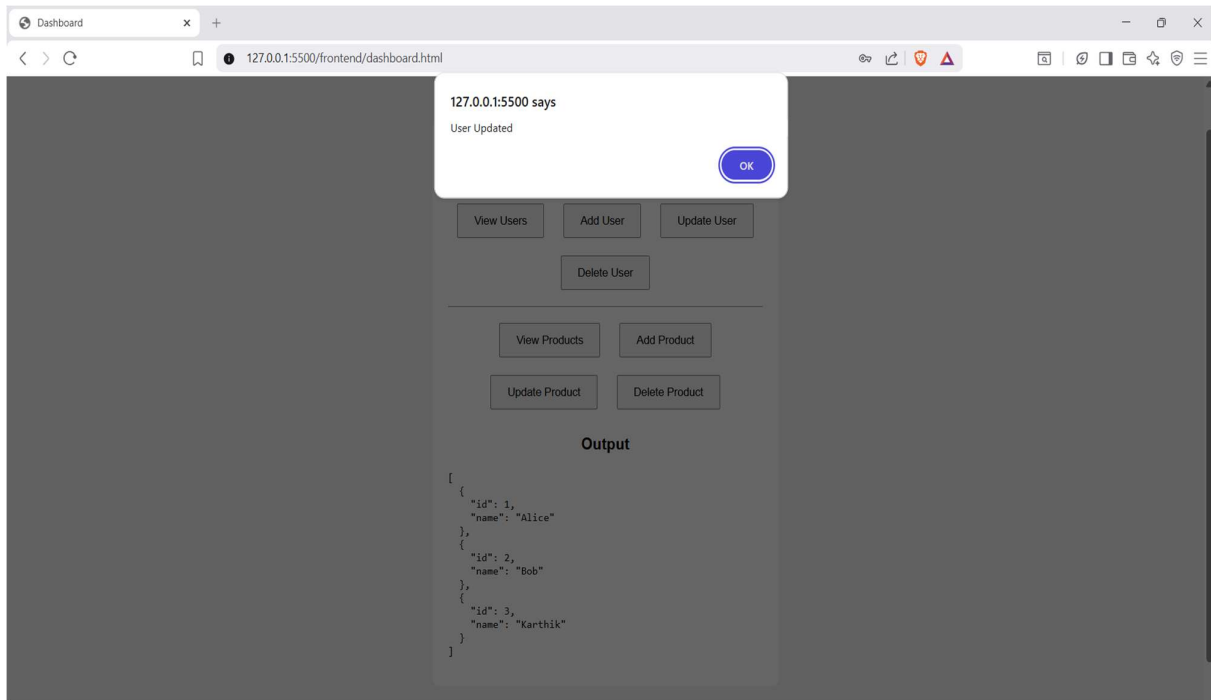
Add User



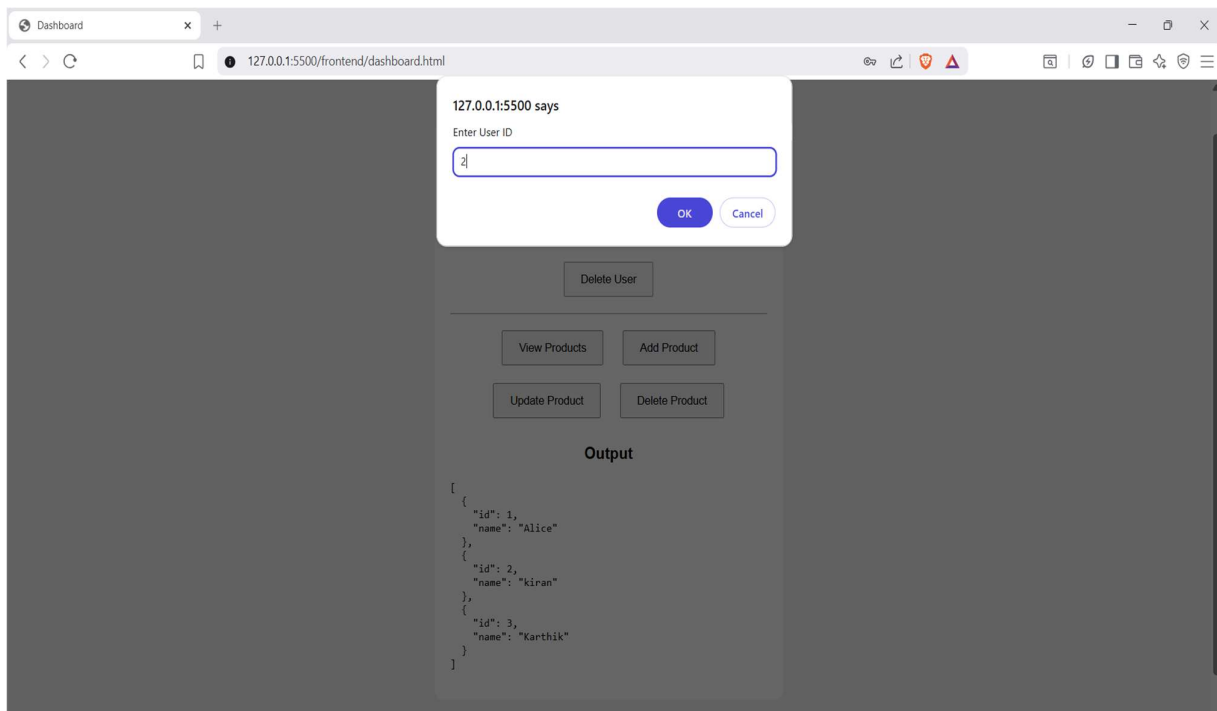
User Added Success full



Update User



User Update Success full



Delete User

Dashboard

View Users

Add User

Update User

Delete User

View Products

Add Product

Update Product

Delete Product

Output

```
[
  {
    "id": 1,
    "name": "Alice"
  },
  {
    "id": 3,
    "name": "Karthik"
  }
]
```

User Deleted

Dashboard

View Users

Add User

Update User

Delete User

View Products

Add Product

Update Product

Delete Product

Output

```
[
  {
    "id": 1,
    "name": "Laptop"
  },
  {
    "id": 2,
    "name": "Phone"
  }
]
```

View Products

CONCLUSION

The project “**Secure User & Product Management System using Microservices**” was successfully designed and implemented. The system provides secure user authentication using JWT tokens and performs CRUD operations for users and products through independent microservices.

The application uses a frontend developed with HTML, CSS, and JavaScript, while backend services were created using Express.js. NGINX was used as an API Gateway to route requests between services efficiently. JWT authentication ensures secure access to protected APIs.

In addition, the project improves understanding of secure communication between frontend and backend systems using token-based authentication. The use of separate services for user management and product management increases flexibility and supports independent development and maintenance. By integrating JWT security, NGINX routing, and CRUD functionalities, the project simulates a real-world web application environment and enhances practical knowledge in full-stack web development and API-based architectures.

The project demonstrates the concepts of:

- Microservices Architecture
- API Gateway Routing
- JWT Authentication
- CRUD Operations
- Frontend and Backend Integration

The developed system is scalable, modular, secure, and easy to maintain. The project also provides practical understanding of modern web application development and service-based architecture.

Finally, the system successfully achieved all the objectives of secure authentication, API communication, and CRUD management using microservices.